

Lecture 5

Managing files and folder permission

CoSc2043 – Network and System Administration

Lecture Overview

- ◆ Types of Files (Regular files, directories)
- ◆ Character and block files
- ◆ File attributes and ownership
- ◆ Link Files (Hard links, symbolic links)
- ◆ Changing access permissions
- ◆ Special attributes
- ◆ Searching the filesystem

Types of Files

Linux defines seven types of files.

No	File type	Symbol	Created by	Removed by
1	Regular file	-	editors, cp, etc.	rm
2	Directory	d	mkdir	rmdir, rm -r
3	Character device file	c	mknod	rm
4	Block device file	b	mknod	rm
5	Local domain socket	s	socket(2)	rm
6	Named pipe	p	mknod	rm
7	Symbolic link	l	In -s	rm

Types of Files (Cont'd)

- ◆ Regular Files :- Most files used directly by a human user are regular files. For example, executable files, text files, and image files are regular files.
- ◆ Directories:- organize folders and files into a hierarchical structure.
- ◆ Devices
 - Character Devices
 - Block Devices
- ◆ Sockets and Named Pipes
- ◆ Symbolic Links : is a pointer or an alias to another file.

Files and Directories

◆ Everything is referenced via a file

◆ Directories

- List of files & inodes
- "." – Reference to the current directory
- ".." – Reference to parent directory
- Root (/) – "." and ".." are the same

◆ Inodes (index node) :- contains details about each file, such as the node, owner, file, location of file

Standard directories and their contents

Pathname	Contents
/bin	Commands needed for minimal system operability
/boot	Kernel and files needed to load the kernel
/dev	Device entries for disks, printers, pseudo terminals, etc.
/etc	Critical startup and configuration files
/home	Home directories for users
/lib	Libraries and parts of the C compiler
/media	Mount points for filesystems on removable media
/opt	Optional application software packages (not yet widely used)
/proc	Information about all running processes
/root	Home directory of the superuser (often just /)
/sbin	Commands for booting, repairing, and recovering the system
/tmp	Temporary files that may disappear between reboots
/usr	Hierarchy of secondary files and commands
/usr/bin	Most commands and executable files
/usr/include	Header files for compiling C programs
/usr/lib	Libraries; also, support files for standard programs
/usr/local	Local software (software you write or install)
/usr/local/bin	Local executables
/usr/local/etc	Local system configuration files and commands

Standard directories and their contents (Cont'd)

Pathname	Contents
/usr/local/bb	Local support files
/usr/local/sbin	Statically linked local system maintenance commands
/usr/local/src	Source code for /usr/local/*
/usr/man	On-line manual pages
/usr/sbin	Less essential commands for system administration and repair
/usr/share	Items that might be common to multiple systems (read-only)
/usr/share/man	On-line manual pages
/usr/src	Source code for nonlocal software packages (not widely used)
/var	System-specific data and configuration files
/var/adm	Varies: logs, system setup records, strange administrative bits
/var/log	Various system log files
/var/spool	Spooling directories for printers, mail, etc.
/var/tmp	More temporary space (preserved between reboots)

Device Files

◆ Character Devices

- Transfer unit: byte
- Example: `/dev/console`

◆ Block Devices

- Transfer unit: Group of bytes (block)
- Examples: `/dev/hda`

◆ Device Numbers

- Major – Type of device
- Minor – Device number

Device Files (Cont'd)

Character Driver	Block Driver
Character devices are accessed as a stream of sequential data, one byte after another.	Block Device are hardware devices which randomly access fixed-sized chunks of data.
No buffering is required.	Accessed through a cache so buffering is required.
Seeking is not allowed.	Seeking is possible.
Managing char driver within a kernel require less care, preparation and work done.	Managing block driver within a kernel requires more care, preparation and work done.
Character driver has only one position current one. It can't move back and forth.	Block driver navigates back and forth between any location on media.
The read() and write() calls do not return until the operation is complete.	Reads and write done by buffer cache mechanism by bread(), bwrite().These request may be asynchronous.
The kernel doesn't have to provide an entire subsystem to the character device.	The kernel does have to provide an entire subsystem to block device.
sources for character devices are kept in .../kernel/chr_drv/	Sources for block devices are kept in .../kernel/blk_drv/
Less complex.	More complex.
Eg: Keyboard device, serial port.	Eg: Hard disk, Pen drive, Blue-rays.

Sockets & Named Pipes

- ◆ Enables communication between processes

- ◆ Socket

- Processed must have a connection first
- Example: X Windows

- ◆ Named Pipe

- Communication between unrelated processes
- FIFO
- Not used very often

Link Files

◆ Multiple names for same file

◆ Hard Link

- Pointer to Inode
- Can't cross partitions
- Equivalent to the original file reference
- File removed when all links deleted

◆ Symbolic (Soft) Links

- Pointer to file path name
- Dangling symlink – *Real* file which no longer exists

◆ `ln [-s] <real_file> <link_file>`

Magic Numbers

- ◆ Byte pattern at beginning of file
- ◆ Patterns listed in file called **magic**
RedHat: `/usr/share/magic`
- ◆ **file** – Tests a file to determine type
 - Filesystem Test
 - Magic Number Test
 - Language Test

MS File/Directory Attributes



UNIX/Linux File Attributes

Inode
Permissions
Ownership
Time Stamps ♦ Change ♦ Modification ♦ Access
File Size
Link Count
Pointers to data

Viewing File Attributes

ls -l: Long listing (includes attributes)

- ◆ File Type
- ◆ File Size/Device #
- ◆ Permissions
- ◆ Modification Date
- ◆ Link Count
- ◆ File Name
- ◆ Ownership

stat: Lists all attributes

File Type Attribute

```
# ls -ld /home /etc/passwd /dev/console
```

```
crw--w--w- 1 root    root      5,   1 Sep 27 11:27 /dev/console
-rw-r--r-- 1 root    root      559 Sep 22 13:14 /etc/passwd
drwxr-xr-x 3 root    root        0 Sep 26 10:42 /home
```

File Type	Meaning
-	Regular File
d	Directory
l	Symbolic Link
b	Block Device
c	Character Device
p	Named Pipe
s	Domain Socket

Ownership

```
# ls -ld /home /etc/passwd /dev/console
crw--w--w-  1 root    root      5,   1 Sep 27 11:27 /dev/console
-rw-r--r--  1 root    root      559 Sep 22 13:14 /etc/passwd
drwxr-xr-x  3 root    root      0 Sep 26 10:42 /home
```

◆ User

- Owner of file
- User names/UIDs defined in `/etc/passwd`

◆ Group

- Organization of users accessing the file
- Group names/GIDs defined in `/etc/group`

Permissions

```
# ls -ld /home /etc/passwd /dev/console
crw--w--w-  1 root    root      5,   1 Sep 27 11:27 /dev/console
-rw-r--r--  1 root    root      559 Sep 22 13:14 /etc/passwd
drwxr-xr-x  3 root    root      0 Sep 26 10:42 /home
```

Operation	File	Directory
Read	Read file	List files
Write	Delete/Modify file	Create/Delete file
Execute	Run program	Access file

◆ 3 levels of access – Owner, Group, Other

Changing Access

◆ Users & Groups

- `chown [-R] user file...`
- `chgrp [-R] group file`

◆ Permissions

- `chmod [-R] <op> file...`
- Numeric: `<op> = [#]###`
- Symbolic: `<op> = <who op perm>`
 - ◆ Who: (u)ser (g)roup (o)ther (a)ll
 - ◆ op: (+)add (-)remove (=)set
 - ◆ Perm: (r)ead (w)rite e(x)ecute

Numeric/Symbolic Permissions

Octal	Binary	Symbolic
0	000	---
1	001	--x
2	010	-w-
3	011	-wx
4	100	r--
5	101	r-x
6	110	rw-
7	111	rwx

Default Permissions

- ◆ **umask** Shell Environment Variable
- ◆ Defines permissions to remove

Numeric	Binary	Effective Perms
0	000	rwX
1	001	rw-
2	010	r-w
3	011	r--
4	100	-wX
5	101	-w-
6	110	--X
7	111	---

Special Attributes

◆ Setuid (SUID) Bit

- Run program with access of owner
- Symbolic: **s**
- Numeric: 4000

◆ Setgid (SGID) Bit

- Run program with access of owner group
- Symbolic: **s**
- Numeric: 2000

Sticky Bit

◆ Purpose

- File: Force program to stay in RAM (obsolete)
- Directory: Cannot remove file unless you own the file or directory

◆ Symbolic: `t`

◆ Numeric: 1000

◆ Example: `/tmp`

Microsoft Permissions

- ◆ Read
- ◆ Create
- ◆ Write
- ◆ Append
- ◆ Delete
- ◆ Execute
- ◆ Search
- ◆ Ownership
- ◆ Access Control

GUI and Command Line (`cac1s`) tools to manage

Searching the Filesystem

find: Command line search tool

- ◆ Searches through directory hierarchy
- ◆ Search by any combination of file names and attributes
- ◆ Display files or perform operations on them
- ◆ Examples:
 - `find /var -mtime -1`
 - `find / -name core -exec rm -f {} \;`